Topic 4: Predicting Categorical Variables

Instructor: Dr. Akash Gupta

Table of contents

1 Classification Problems

Classification problems in predictive analytics involve predicting a discrete class label for an input based on its features, using a labeled dataset for training. They

are divided into binary classification (two classes, e.g., "spam" or "not spam") and multi-class classification (multiple classes, e.g., "cat," "dog," "bird").

Examples:

- **Direct marketing:** Predict whether a customer will give a positive or a negative response to a campaign
- **Medicine:** Predict whether a patient is healthy or sick
- **Insurance:** Classify clients by risk level; for instance, low, average, or high risk
- **Telecommunication and other industries:** Churn models are classification models that predict which customers will switch to another provider
- **Education:** Predict which students will drop out from a program

Classification models can output two types of predictions:

- **Predicted classes:** For every observation, the model will directly give the prediction of the class. Example,

```
predicted_classes = ['Not Spam', 'Spam', 'Not Spam']
```

- **Probabilities for each class:** For every observation and every class, the model will output probabilities of that observation belonging to that class. For example,

```
probabilities = [
{'Not Spam': 0.85, 'Spam': 0.15},
{'Not Spam': 0.30, 'Spam': 0.70},
{'Not Spam': 0.95, 'Spam': 0.05}
]
```

2 Logistic Regression

Logistic regression is a statistical method for predicting the outcome of a binary variable based on one or more predictor variables. It models the probability that a given input point belongs to a certain category.

2.1 Mathematical Explanation

Logistic Function: The logistic function, or sigmoid function, is central to logistic regression and is defined as:

$$P(Y = 1|X) = \frac{1}{1 + e^{-z}}$$

where z is the linear combination of input features x_i, weighted by coefficients β_i, i.e., $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$.

Substitute (z):

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

Let:

$$p = P(Y = 1 \mid X)$$

$$p = \frac{1}{1 + e^{-z}}$$

$$e^{-z} = \frac{1 - p}{p}$$

$$z = \ln\left(\frac{p}{1 - p}\right)$$

Substitute back for (z):

$$\ln\left(\frac{p}{1 - p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Odds Ratio: The odds of the outcome are the ratio of the probability of the event occurring to the probability of the event not occurring:

$$\text{Odds} = \frac{P(Y = 1|X)}{1 - P(Y = 1|X)} = e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n}$$

Logit Function: The logit function is the logarithm of the odds ratio and is a linear combination of the input features:

$$\ln\left(\frac{P(Y=1|X)}{1-P(Y=1|X)}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

2.2 Model Estimation

The coefficients β_i are estimated using maximum likelihood estimation (MLE), which seeks to maximize the likelihood function given the observed data.

2.3 Interpretation

- **Coefficients (β_i):** Indicate the change in the log odds of the outcome for a one-unit increase in the predictor x_i, holding all other predictors constant.
- **Odds Ratio:** e^{β_i} represents the factor by which the odds change for a one-unit increase in x_i. If $e^{\beta_i} > 1$, the odds of the outcome increase; if $e^{\beta_i} < 1$, the odds decrease.

Example:

Suppose we want to predict whether a patient has diabetes (1) or not (0) based on:

- (X_1) = Age (given coefficient = 0.04)
- (X_2) = BMI (given cofficient = 0.08)

$$\log\left(\frac{P}{1-P}\right) = -5 + 0.04(Age) + 0.08(BMI)$$

Interpretation:

- 1 unit increase in Age increases log-odds by 0.04
- 1 unit increase in BMI increases log-odds by 0.08

Logistic regression converts (z) into a probability using the logistic (sigmoid) function:

$$P(Y=1|X) = \frac{1}{1+e^{-z}}$$

Substitute the equation for (z):

$$P(Y = 1|X) = \frac{1}{1 + e^{-(-5+0.04Age+0.08BMI)}}$$

- Age = 50
- BMI = 30

First compute (z):

$$z = -5 + 0.04(50) + 0.08(30)$$

$$z = -5 + 2 + 2.4 = -0.6$$

Now compute probability:

$$P(Y = 1) = \frac{1}{1 + e^{0.6}}$$

$$P(Y = 1) \approx 0.35$$

So the model predicts a 35% probability of diabetes.

3 Classification Tree

- Classification trees are a type of decision tree used for separating the dataset into classes based on feature variables.
- They work by iteratively splitting the data based on specific features, creating a tree-like model of decisions.
- Its non-parametric approach allows for flexible modeling without assuming the distribution of input variables.

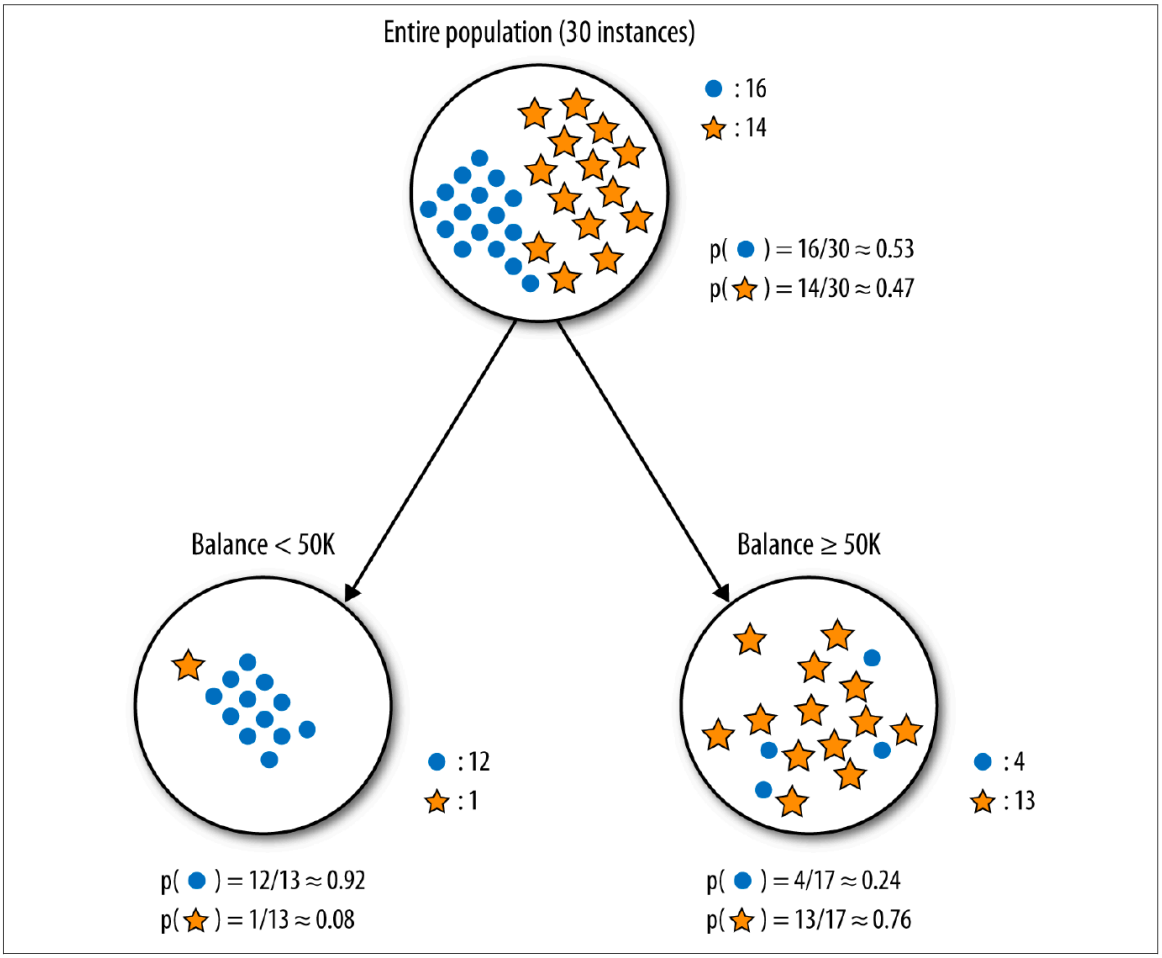


Figure 1: Splitting in classification tree

3.1 How Classification Trees Work

- **Starting at the Root:** The entire dataset is considered as the root.
- **Feature Splits:** The dataset is split on the feature that results in the most significant information gain (IG).
- **Recursion:** Steps are recursively applied to each split until one of the termination conditions is met.

In a classification tree, feature splits are used to recursively partition the dataset into increasingly homogeneous subsets based on target variables.

- Information Gain
- Ginni Index

3.2 Entropy

Entropy is a measure of disorder or impurity in a set of data. In the context of decision trees (a common supervised learning method), entropy helps to determine which attribute or feature should be tested at a node. The goal is to choose the attribute that provides the most effective way to split the data into subsets that have a clear majority of one class or another.

The mathematical formula for entropy of a dataset $H(S)$ with respect to a binary classification is:

$$H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

Where:

- p_1 is the proportion of positive examples in S
- p_2 is the proportion of negative examples in S

For multi-class classification problems, the entropy is extended as:

$$H(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Where:

- n is the number of classes
- p_i is the proportion of examples that belong to class i

Example: Consider a set S of 10 people with seven of the non-write-off class and three of the write-off class. Compute entropy ?

Given the proportions of the write-off and non-write-off classes:

$$p_1 = \frac{3}{10}$$

$$p_2 = \frac{7}{10}$$

The entropy for the set S is calculated using the formula:

$$H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

Plugging in the given values:

$$H(S) = -\frac{3}{10} \log_2 \left(\frac{3}{10}\right) - \frac{7}{10} \log_2 \left(\frac{7}{10}\right) \approx 0.8813$$

Thus, the entropy $H(S)$ is approximately 0.8813.

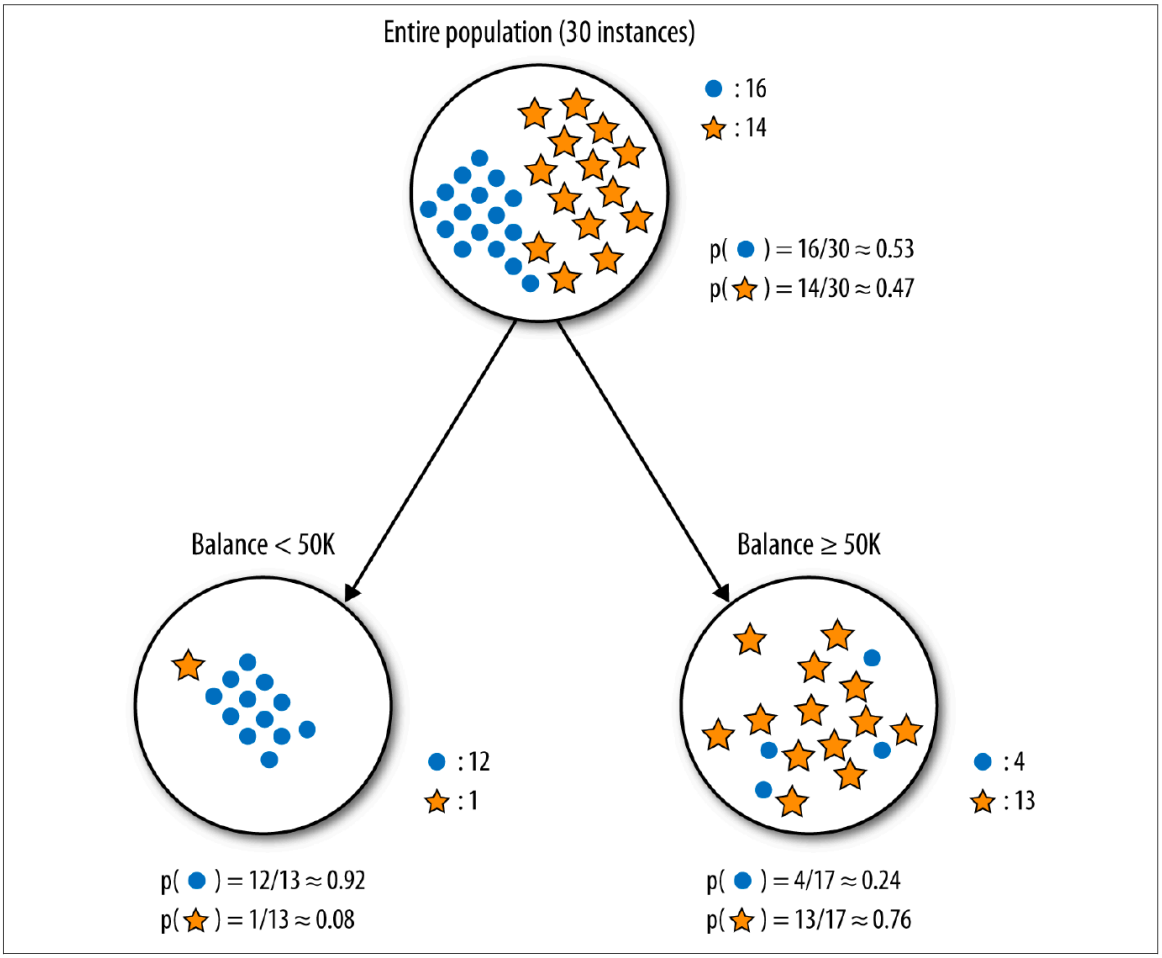
3.3 Information Gain

Information Gain $=$ Entropy(parent)$-[p(c_1) \times$ Entropy$(c_1) + p(c_2) \times$ Entropy$(c_2)]$

Where:

- **Entropy(parent)**: Represents the entropy of the original dataset or the parent node, quantifying its randomness or unpredictability.
- $p(c_1)$: The proportion of examples in the first subset resulting from the split.
- $p(c_2)$: The proportion of examples in the second subset resulting from the split.
- **Entropy(c_1)**: The entropy of the first subset after the split, which may contain positive or mixed examples.
- **Entropy(c_2)**: The entropy of the second subset after the split, which may contain negative or mixed examples.

Example: Compute information gain in the following figure.



$$H(\text{Parent}) = -\left(\frac{16}{30}\log_2\frac{16}{30} + \frac{14}{30}\log_2\frac{14}{30}\right) = 0.997$$

- **Child 1**: 1 star, 12 round (Total = 13)

$$H(\text{Child 1}) = -\left(\frac{12}{13}\log_2\frac{12}{13} + \frac{1}{13}\log_2\frac{1}{13}\right) = 0.391$$

- **Child 2**: 4 round, 13 star (Total = 17)

$$H(\text{Child 2}) = -\left(\frac{4}{17}\log_2\frac{4}{17} + \frac{13}{17}\log_2\frac{13}{17}\right) = 0.787$$

The weighted sum of the child entropies is computed as:

$$\text{Weighted Sum} = \frac{13}{30}H(\text{Child 1}) + \frac{17}{30}H(\text{Child 2}) = 0.616$$

$$\text{Information Gain}(IG) = H(\text{Parent}) - \text{Weighted Sum} = 0.381$$

Practice Problem: Compute information gain in the following figure.

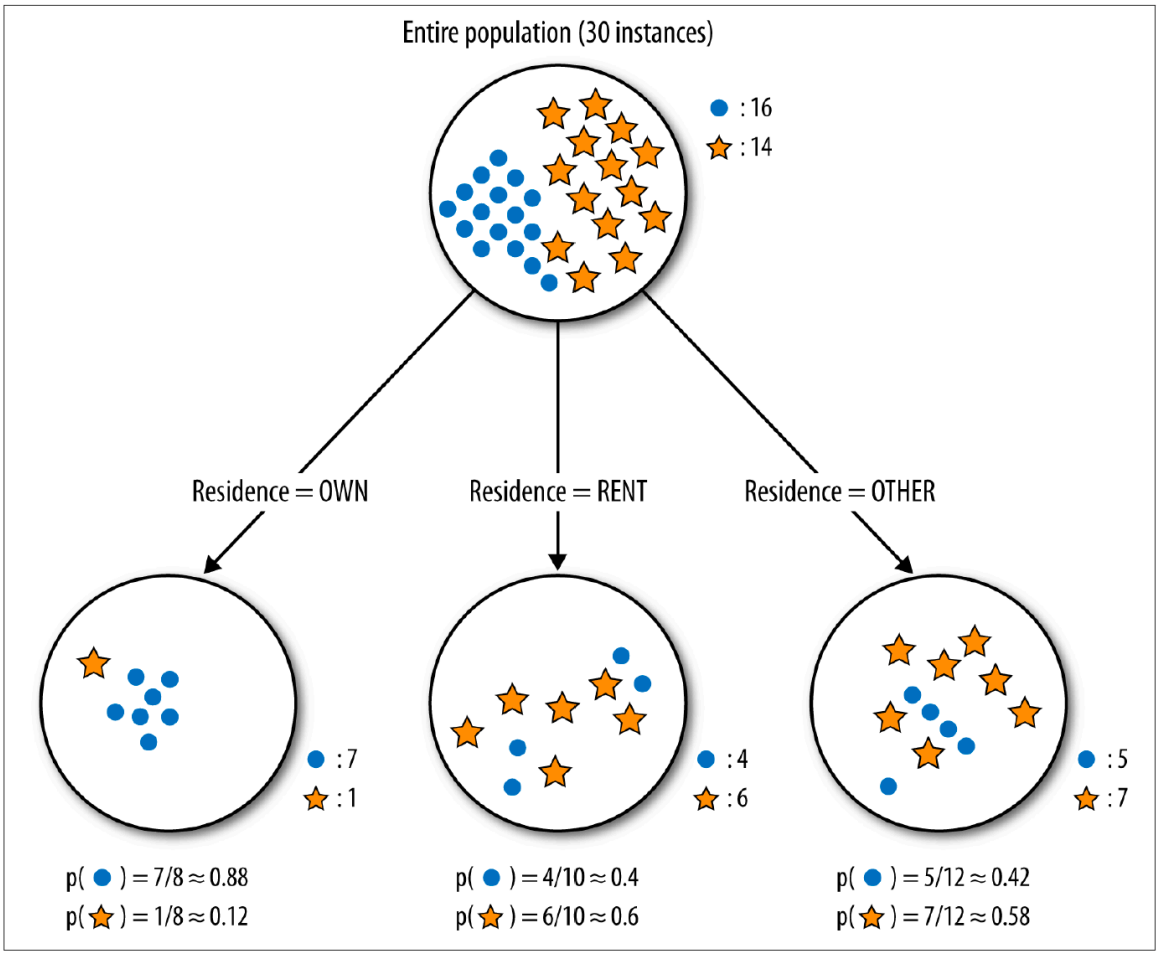


Figure 2: Practice example

Answer: Information Gain: 0.13 (approx.)

3.4 Ginni Index

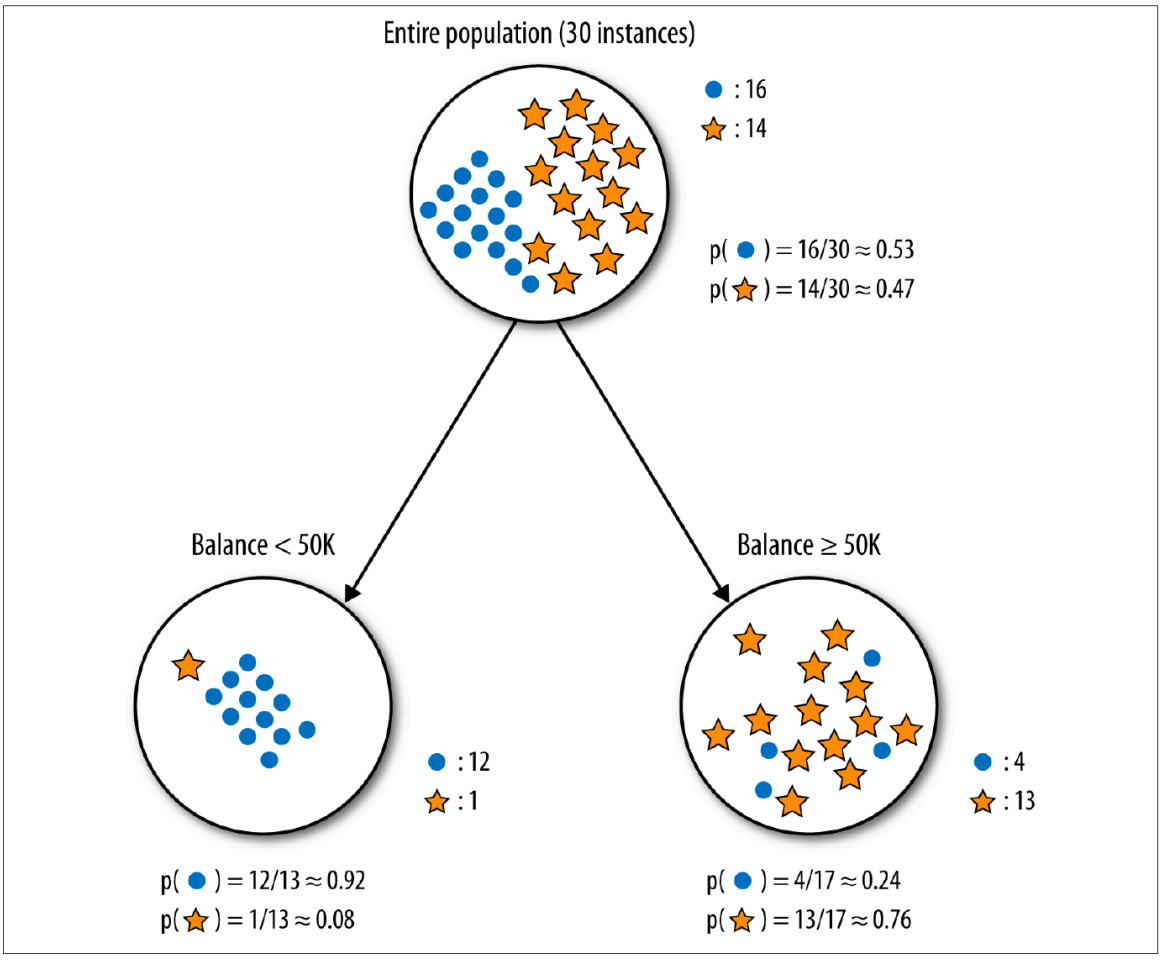
Gini index is defined as:

$$Gini(D) = 1 - \sum_{i=1}^{C} p_i^2$$

where (p_i) is the proportion of instances in class (i) for a dataset (D) containing (C) classes.

Ginni index measures how often a randomly chosen observation would be misclassified. Lower Gini leads to purer node.

Example: Compute Ginni index in the following figure.



$$Gini(\text{Parent}) = 1 - \left(\left(\frac{16}{30} \right)^2 + \left(\frac{14}{30} \right)^2 \right) = 0.498$$

- For Child 1 (12 round, 1 star):

$$Gini(\text{Child 1}) = 1 - \left(\left(\frac{12}{13} \right)^2 + \left(\frac{1}{13} \right)^2 \right) = 0.142$$

- For Child 2 (4 round, 13 star):

$$Gini(\text{Child 2}) = 1 - \left(\left(\frac{4}{17} \right)^2 + \left(\frac{13}{17} \right)^2 \right) = 0.360$$

$$\text{Weighted Gini} = \frac{13}{30} Gini(\text{Child 1}) + \frac{17}{30} Gini(\text{Child 2}) = 0.265$$

Example

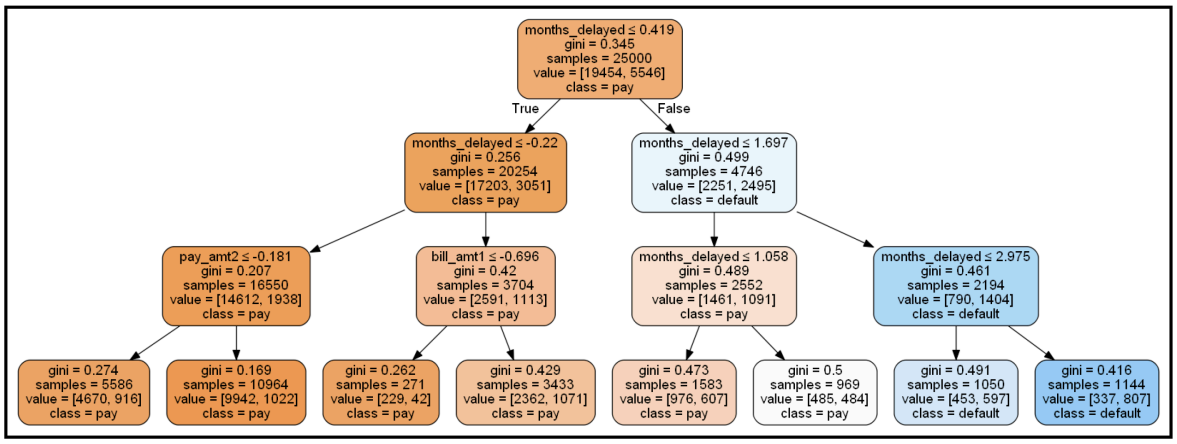


Figure 3: Decision Tree Example

4 Random Forest

- Random Forest is an ensemble learning method that constructs multiple decision trees
- By aggregating the predictions of numerous trees, Random Forest achieves higher performance and robustness than individual decision trees.

4.1 How Random Forest Work

- **Bootstrap Samples:** For a dataset of size N, Random Forest starts by creating n new training datasets of size N', each created by randomly selecting samples from the original dataset with replacement.
- **Aggregating or Bagging:** After training on these bootstrap samples, Random Forest aggregates their predictions. For classification, it uses majority voting. For regression, it calculates the average of the predictions.

Each tree in the forest is constructed based on the following principles:

- At each node, rather than evaluating all features for the best split, a random subset of features is selected. This introduces randomness, aiding in the reduction of correlation among the trees.
- The optimal split at each node is determined using a criterion such as Gini Impurity or Information Gain, akin to the methodology in a singular decision tree.

4.2 Feature Importance

Random Forest provides insights into the importance of features through the following approach:

- The significance of a feature is assessed by examining the extent to which tree nodes utilizing that feature contribute to the reduction of impurity across the entire forest.
- These contributions to impurity reduction are averaged across all trees and normalized to ensure their sum equals 1.

5 Naive Bayes

The core of the Naive Bayes model is Bayes' theorem, which describes the probability of an event, based on prior knowledge of conditions that might be related to the event. Mathematically, it's expressed as:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

where:

- $P(A|B)$ is the posterior probability of class A given predictor B,
- $P(B|A)$ is the likelihood which is the probability of predictor given class,
- $P(A)$ is the prior probability of class,
- and $P(B)$ is the prior probability of predictor.

In the context of Bayesian statistics, the components of Bayes' theorem are described with specific terminology:

- $P(\text{A}|\text{B})$: **Posterior probability** or simply **posterior**, represents the probability of the event of interest (defaulting) after considering some specific information (the customer is male).
- $P(\text{A})$: **Prior probability** or simply **prior**, is the probability of the event of interest (defaulting) before considering any specific information.
- $P(\text{B}|\text{A})$: **Likelihood**, the conditional probability of observing our specific information (the customer is male) given that the event of interest (defaulting) is true.
- $P(\text{B})$: The probability of observing the specific information (the customer is male), sometimes referred to as the **probability of the evidence**.

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

6 Case Study: Credit Card Default Dataset

6.1 Problem Statement and Data

When a customer accepts a credit card from a bank or issuer, he/she agrees to certain terms and condition such as he/she needs to make their minimum payment by the due date listed on their credit card statements. If the consumer fails to make the payment for the debt by the due date, the issuer mark the credit card as default and might charge a penalty, decreasing credit limit and and in case of serious delniquency, close the account.

Sometimes credit card-issuing banks in order to gain large amount of market share issue credit cards to unqualified clients without suffient information about their ability of repayment of the bills. When the card holders overuse their cards to consume servises and goods irrespective of their repayment ability of the bills, they accumulate a heavy debts.

In the area of consumer finance it is pivotal for the card issuing banks to be able to estimate the chance for a card holder to become default for risk analysis and approving the credit card applications.

- **SEX:** Gender (1 = male; 2 = female).
- **EDUCATION:** Education (1 = graduate school; 2 = university; 3 = high school; 4 = others).
- **MARRIAGE:** Marital status (1 = married; 2 = single; 3 = others).
- **AGE:** Age (year).
- **LIMIT_BAL:** Amount of the given credit (New Taiwan dollar)—it includes both the individual consumer credit and his/her family (supplementary) credit.
- **PAY_1 - PAY_6:** History of past payment. We tracked the past monthly payment records (from April, 2005, to September, 2005) as follows: 0 = the repayment status in September, 2005; 1 = the repayment status in August, 2005; ...; 6 = the repayment status in April, 2005. The measurement scale for the repayment status is: -1 = pay duly; 1 = payment delay for one month; 2 = payment delay for two months; ...; 8 = payment delay for eight months; 9 = payment delay for nine months and above.
- **BILL_AMT1-BILL_AMT6:** Amount of bill statement (New Taiwan dollar). BILL_AMT1 = amount of bill statement in September, 2005; BILL_AMT2 = amount of bill statement in August, 2005; ...; BILL_AMT6 = amount of bill statement in April, 2005.
- **PAY_AMT1-PAY_AMT6:** Amount of previous payment (New Taiwan dollar).

6.2 Python Code

6.2.1 Data Importing

```
## Data Loading
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve

data_path = '/Users/akashgupta/Library/CloudStorage/Box-Box/Teaching/MasterCopy/BANA_620_Mast
# Load the dataset into a pandas DataFrame and set the index to the "ID" column
ccd = pd.read_csv(data_path, index_col="ID")

# Understand the data
ccd.describe()
```

6.2.2 Initial Processing

```
# Clean PAY_ features
pay_features = ['PAY_' + str(i) for i in range(1,7)]

# Loop through each 'pay_' feature
for x in pay_features:
# Transform the -1 and -2 values to 0, assuming these represent non-defaulted payments
  ccd.loc[ccd[x] <= 0, x] = 0

# Rename the 'default payment next month' column to 'default' for brevity
ccd.rename(columns={'default payment next month':'default'}, inplace=True)
categorical_features = ['SEX', 'EDUCATION', 'MARRIAGE']
numerical_features = ['LIMIT_BAL', 'AGE', 'PAY_1', 'PAY_2', 'PAY_3', 'PAY_4', \
    'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', \
        'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4',\
            'PAY_AMT5', 'PAY_AMT6']
target_feature = ['default']
```

6.2.3 Standardize Numeric Features and Create Dummy Variables

```
scaler = StandardScaler()
ccd[numerical_features] = scaler.fit_transform(ccd[numerical_features])
ccd = pd.get_dummies(ccd, columns=categorical_features, drop_first=True)
```

6.2.4 Splitting Data into Train and Test

```
X = ccd.drop(columns=['default'])
y = ccd['default']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

6.2.5 Building Models

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_train_pred = rf_model.predict(X_train)
rf_test_pred = rf_model.predict(X_test)
rf_test_proba = rf_model.predict_proba(X_test)[:, 1]

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_train_pred = dt_model.predict(X_train)
dt_test_pred = dt_model.predict(X_test)
dt_test_proba = dt_model.predict_proba(X_test)[:, 1]

lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)
lr_train_pred = lr_model.predict(X_train)
lr_test_pred = lr_model.predict(X_test)
lr_test_proba = lr_model.predict_proba(X_test)[:, 1]
```

6.2.6 Model Performance Comparison

```
models = ['Random Forest', 'Decision Tree', 'Logistic Regression']
train_accuracies = [
    accuracy_score(y_train, rf_train_pred),
    accuracy_score(y_train, dt_train_pred),
    accuracy_score(y_train, lr_train_pred)
]
test_accuracies = [
    accuracy_score(y_test, rf_test_pred),
```

```
    accuracy_score(y_test, dt_test_pred),
    accuracy_score(y_test, lr_test_pred)
]
auroc_scores = [
    roc_auc_score(y_test, rf_test_proba),
    roc_auc_score(y_test, dt_test_proba),
    roc_auc_score(y_test, lr_test_proba)
]


results_df = pd.DataFrame({
    'Model': models,
    'Training Accuracy': [f"{x:.4f}" for x in train_accuracies],
    'Testing Accuracy': [f"{x:.4f}" for x in test_accuracies],
    'AUROC Score': [f"{x:.4f}" for x in auroc_scores]
})

# Print the table
print("Model Performance Comparison:")
print(results_df)
```

Model	Training Accuracy	Testing Accuracy	AUROC Score
Random Forest	0.9994	0.8150	0.7620
Decision Tree	0.9995	0.7268	0.6145
Logistic Regression	0.8193	0.8162	0.7631

1. **Training Accuracy: How Well the Model Fits the Training Data**
    - **Random Forest (0.9994) and Decision Tree (0.9995)**: These are *extremely high*—almost perfect! This means these models are learning the training data very well, possibly too well. When you see numbers this high, it's a hint that the model might be memorizing the data (called *overfitting*), especially if testing accuracy is much lower.
    - **Logistic Regression (0.8193)**: This is lower, which might seem "worse," but it's actually more realistic. It suggests the model isn't overfitting and is capturing general patterns rather than every detail of the training set.
2. **Testing Accuracy: How Well the Model Generalizes**
    - **Random Forest (0.8150)**: Decent performance on unseen data, but the big drop from 0.9994 shows overfitting. The model isn't as good at

predicting new data as it is at "remembering" the training data.
- **Decision Tree (0.7268)**: The lowest here, with an even bigger drop from 0.9995. This is a classic sign of overfitting—Decision Trees can grow too complex and fit noise in the training data, making them struggle on the test set.
- **Logistic Regression (0.8162)**: Almost identical to its training accuracy! This consistency is a good sign—it means the model generalizes well and isn't overfitting. This shows that simpler models can sometimes outperform complex ones on new data.

3. **AUROC Score: Measuring Class Separation**

   - **Random Forest (0.7620)**: A decent score, meaning it can distinguish between classes fairly well, but not amazing.
   - **Decision Tree (0.6145)**: The lowest AUROC, close to 0.5 (random guessing).
   - **Logistic Regression (0.7631)**: Similar to Random Forest, but with better consistency across training and testing accuracy. This suggests it's producing reliable probability estimates, which is a strength of this simpler model.

4. **Key Lessons**

   - **Simplicity Wins**: Logistic Regression, despite lower training accuracy, performs best on the test set and has a solid AUROC. This shows that simpler models can be more robust—don't always chase complexity!
   - **Metrics Matter**: Accuracy alone doesn't tell the whole story. AUROC adds insight into how well the model separates classes, especially useful if your data has imbalanced classes (e.g., more 0s than 1s).
   - **Next Steps**: Try plotting the probability distributions (e.g., with histograms) to see how these models assign confidence. Decision Tree might show extreme 0s and 1s, while Random Forest and Logistic Regression might be smoother.

6.2.7 Printing Decision Tree

```
plt.figure(figsize=(12, 8))
plot_tree(dt_model, max_depth=2, class_names=['No Default', 'Default'],filled=True)
plt.show()
```

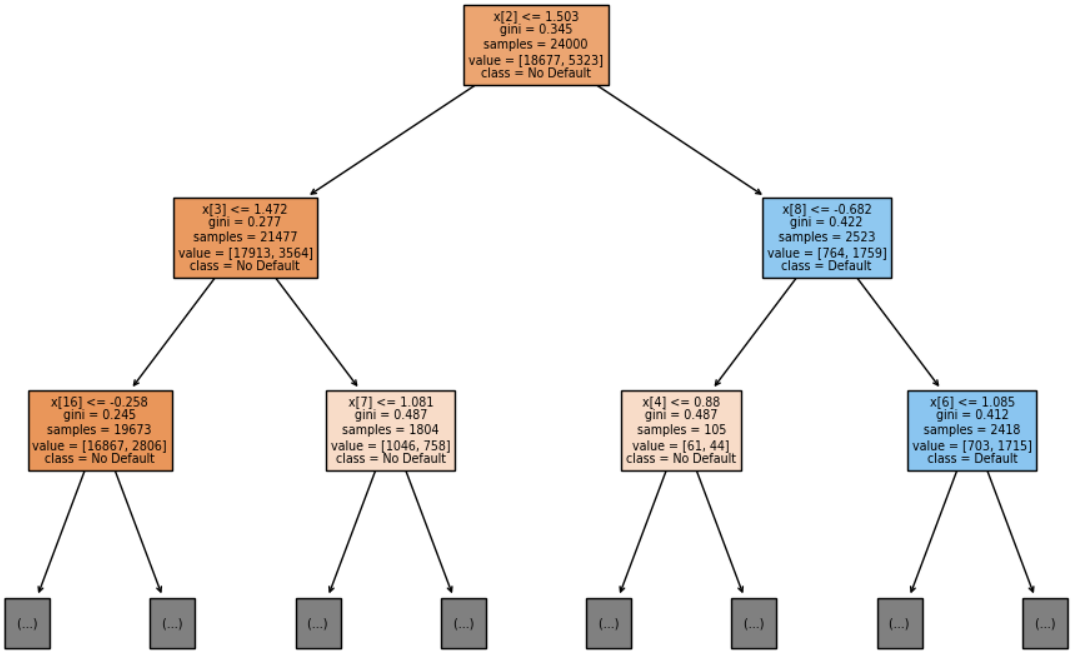


Figure 4: alt text

```
# Calculate ROC curves
rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_test_proba)
dt_fpr, dt_tpr, _ = roc_curve(y_test, dt_test_proba)
lr_fpr, lr_tpr, _ = roc_curve(y_test, lr_test_proba)

# ax3 = plt.plot(1, 1, 1)
plt.plot(rf_fpr, rf_tpr, label=f'Random Forest (AUC = {auroc_scores[0]:.3f})', color='blue')
plt.plot(dt_fpr, dt_tpr, label=f'Decision Tree (AUC = {auroc_scores[1]:.3f})', color='orange
plt.plot(lr_fpr, lr_tpr, label=f'Logistic Regression (AUC = {auroc_scores[2]:.3f})', color='g
plt.plot([0, 1], [0, 1], 'k--')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

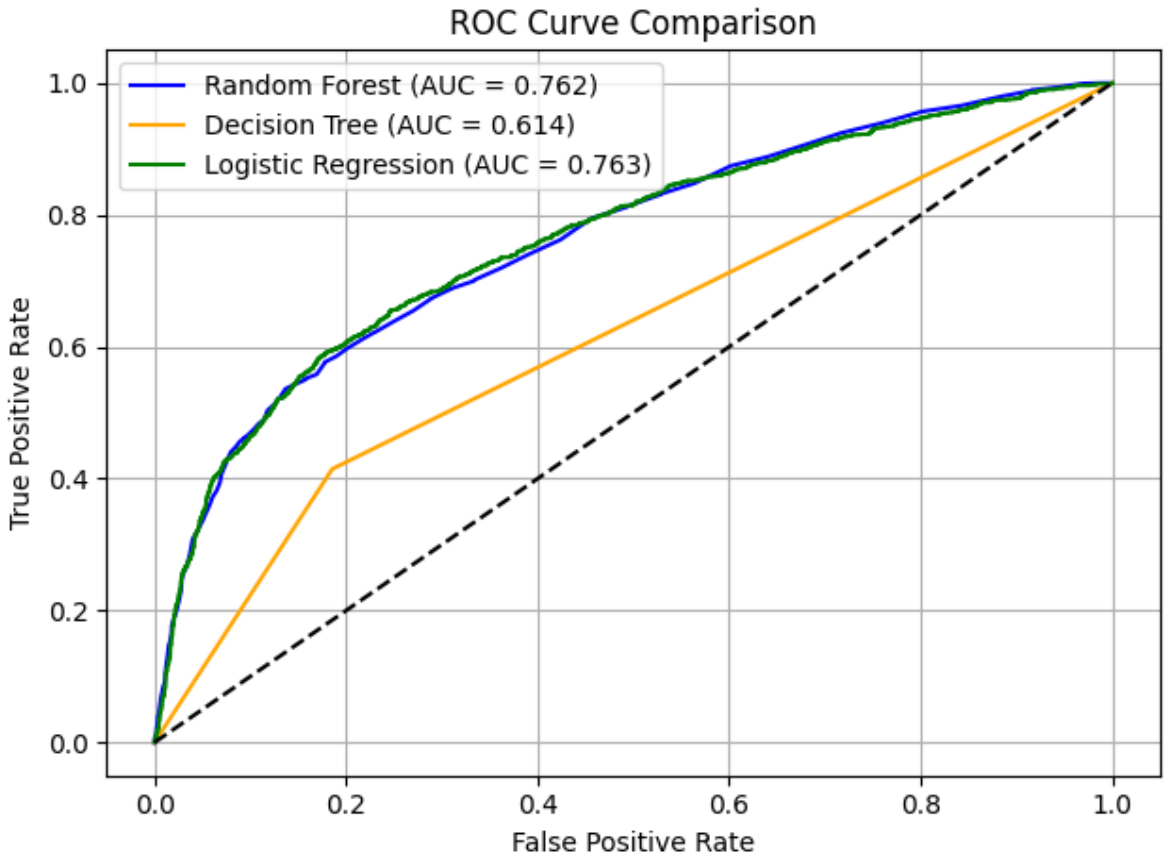
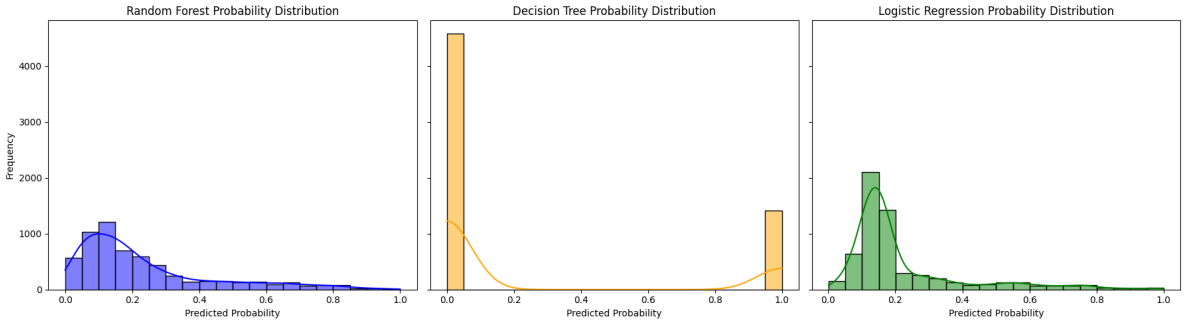


Figure 5: Receiver operating characteristic curve

```
# Create probability distribution plots
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

# Random Forest
sns.histplot(rf_test_proba, bins=20, kde=True, ax=axes[0], color='blue')
axes[0].set_title('Random Forest Probability Distribution')
axes[0].set_xlabel('Predicted Probability')
axes[0].set_ylabel('Frequency')

# Decision Tree
sns.histplot(dt_test_proba, bins=20, kde=True, ax=axes[1], color='orange')
```

```
axes[1].set_title('Decision Tree Probability Distribution')
axes[1].set_xlabel('Predicted Probability')

# Logistic Regression
sns.histplot(lr_test_proba, bins=20, kde=True, ax=axes[2], color='green')
axes[2].set_title('Logistic Regression Probability Distribution')
axes[2].set_xlabel('Predicted Probability')

plt.tight_layout()
plt.show()
```



Reference

https://analytics-nuts.github.io/Comparative-Study-of-Classification-Techniques-on-Credit-Defaults/

https://bradzzz.gitbooks.io/ga-dsi-seattle/content/dsi/dsi_05_classification_databases/2.1-lesson/assets/datasets/DefaultCreditCardClients_yeh_2009.pdf